

ApexPlanet Software Pvt. Ltd.

INTERNSHIP TASK REPORT

Task 3: Web Application Security

Timeline: Days 25 - 36

Domain: Cybersecurity & Ethical Hacking

Submitted by: Akshat Arin Mazumdar

College: Indore Institute of Science and Technology

Degree: B.Tech

Date: 24 September 2026

1. Objective

The objective of Task 3 was to identify and exploit common OWASP Top 10 web application vulnerabilities within a controlled, legal lab environment - Damn Vulnerable Web Application (DVWA) - in order to understand real-world attack techniques and their corresponding defensive measures. The scope defined for this task covered SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), File Inclusion attacks, advanced usage of Burp Suite, and web security header analysis.

Note: This report documents the sub-tasks that were completed and evidenced with screenshots during this session - Command Injection exploitation, automated OS command injection testing with Commix, and CSRF request analysis using Burp Suite. The remaining sub-tasks (SQL Injection, XSS, File Inclusion, and Security Header analysis) are part of the same Task 3 scope and are planned to be completed and documented in a follow-up submission.

2. Tools Used

Tool	Purpose in this Task
DVWA	Damn Vulnerable Web Application - an intentionally vulnerable PHP/MySQL web app used as the legal, controlled target for practicing OWASP Top 10 exploitation techniques.
Netcat (nc)	Used to set up a listener and receive a shell connection resulting from successful OS command injection.
Commix	Automated OS Command Injection Exploiter used to detect and exploit command injection vulnerabilities in the DVWA target endpoint.
Burp Suite	Web proxy used to intercept, inspect, and analyse HTTP requests - here used to capture a CSRF-vulnerable password-change request.
Git	Used to clone the Commix tool repository from GitHub for local installation.

3. Methodology

3.1 Command Injection Exploitation (Manual)

- Identified DVWA's Command Injection module as the target input field.
- Started a Netcat listener on the attacking machine (nc -lvp 1234) to receive an incoming shell.
- Injected a payload into the vulnerable field to chain a reverse-shell command with the application's intended ping command.
- Upon receiving the connection, used standard shell commands (ls, cd, cat) to browse the DVWA directory structure and read local files such as help.php.

3.2 Automated Command Injection Testing (Commix)

- Cloned the Commix tool from its official GitHub repository and reviewed its usage options.

- Targeted the DVWA Command Injection endpoint (/DVWA/vulnerabilities/exec/) using Commix's -u flag.
- Corrected an initial file-path/target configuration error before re-running the scan.
- Allowed Commix to test the connection, handle the session cookie, and attempt to identify an injectable parameter automatically.

3.3 CSRF Request Analysis (Burp Suite)

- Configured Burp Suite as an intercepting proxy between the browser and the DVWA application.
- Navigated to the DVWA CSRF module and submitted a password-change request while Intercept was switched on.
- Captured and inspected the raw GET request in Burp's Inspector panel, examining query parameters, cookies, and headers.
- Confirmed the absence of an anti-CSRF token in the request, which is the underlying weakness that a CSRF attack exploits.

4. Findings / Results

The following screenshots document the practical execution of the completed sub-tasks, captured directly during the exercise.

```

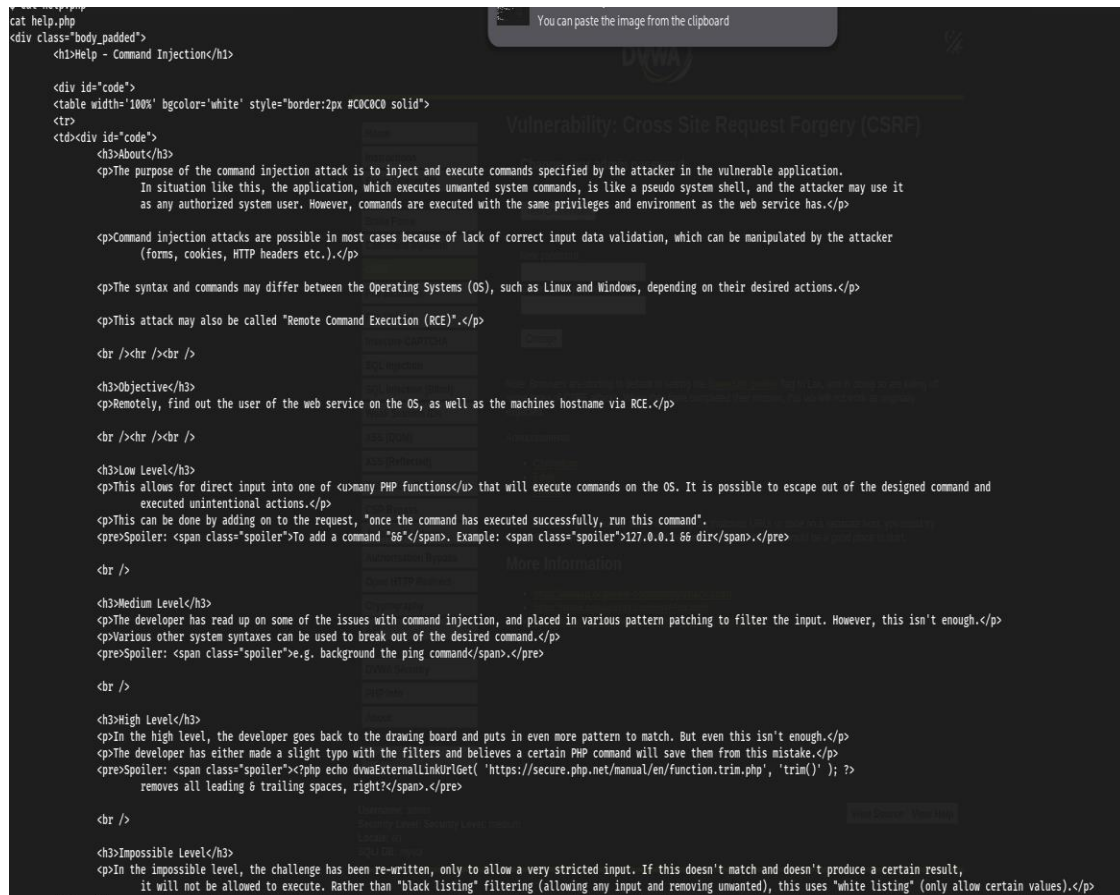
(akshat@kali)-[~]
$ nc -lvnp 1234
listening on [any] 1234 ...
^C

(akshat@kali)-[~]
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [127.0.0.1] from (UNKNOWN) [127.0.0.1] 54348
sh: 0: can't access tty; job control turned off
$ ls
ls
help index.php source
$ cd help
cd help
$ ls
ls
help.php
$ cat help.php
cat help.php
<div class="body_padded">
  <h1>Help - Command Injection</h1>

  <div id="code">
    <table width='100%' bgcolor='white' style="border:2px #C0C0C0 solid">
      <tr>
        <td><div id="code">
  
```

Figure 1: Command Injection - Reverse Shell Access via DVWA

A local listener was started using Netcat (`nc -lvnp 1234`), and a connection was received from the DVWA Command Injection module (127.0.0.1). Basic shell commands (`ls`, `cd`, `cat`) were then used to navigate the DVWA application directory and view the source of `help.php`, demonstrating that OS-level command execution was achievable through the vulnerable input field.



```

cat help.php
<div class="body_padded">
  <h1>Help - Command Injection</h1>

  <div id="code">
    <table width="100%" bgcolor="white" style="border:2px #C0C0C0 solid">
      <tr>
        <td>
          <div id="code">
            <h3>About</h3>
            <p>The purpose of the command injection attack is to inject and execute commands specified by the attacker in the vulnerable application. In situation like this, the application, which executes unwanted system commands, is like a pseudo system shell, and the attacker may use it as any authorized system user. However, commands are executed with the same privileges and environment as the web service has.</p>

            <p>Command injection attacks are possible in most cases because of lack of correct input data validation, which can be manipulated by the attacker (forms, cookies, HTTP headers etc.).</p>

            <p>The syntax and commands may differ between the Operating Systems (OS), such as Linux and Windows, depending on their desired actions.</p>

            <p>This attack may also be called "Remote Command Execution (RCE)".</p>

            <br />
            <h3>Objective</h3>
            <p>Remotely, find out the user of the web service on the OS, as well as the machines hostname via RCE.</p>

            <br />
            <h3>Low Level</h3>
            <p>This allows for direct input into one of <code>many PHP functions</code> that will execute commands on the OS. It is possible to escape out of the designed command and executed unintentional actions.</p>
            <p>This can be done by adding on to the request, "once the command has executed successfully, run this command".</p>
            <pre>Spoiler: <code><span class="spoiler"><code>?php echo `66`</code></span>. Example: <code><span class="spoiler">127.0.0.1 66 dir</span></code></pre>

            <br />
            <h3>Medium Level</h3>
            <p>The developer has read up on some of the issues with command injection, and placed in various pattern patching to filter the input. However, this isn't enough.</p>
            <p>Various other system syntaxes can be used to break out of the desired command.</p>
            <pre>Spoiler: <code><span class="spoiler">e.g. background the ping command</span></code></pre>

            <br />
            <h3>High Level</h3>
            <p>In the high level, the developer goes back to the drawing board and puts in even more pattern to match. But even this isn't enough.</p>
            <p>The developer has either made a slight typo with the filters and believes a certain PHP command will save them from this mistake.</p>
            <pre>Spoiler: <code><span class="spoiler"><code>?php echo dywaExternalLinkUrlGet( 'https://secure.php.net/manual/en/function.trim.php', 'trim()' ); ?</code></span> removes all leading & trailing spaces, right?</code></pre>

            <br />
            <h3>Impossible Level</h3>
            <p>In the impossible level, the challenge has been re-written, only to allow a very strict input. If this doesn't match and doesn't produce a certain result, it will not be allowed to execute. Rather than "black listing" filtering (allowing any input and removing unwanted), this uses "white listing" (only allow certain values).</p>

```

Figure 2: DVWA Command Injection - Built-in Documentation Review

The contents of DVWA's own help.php file were reviewed, which describes the Command Injection vulnerability class in detail - explaining how unsanitised user input can be chained with OS commands (e.g., using '&&') and how the vulnerability's difficulty increases across DVWA's Low, Medium, High, and Impossible security levels through progressively stronger input filtering.

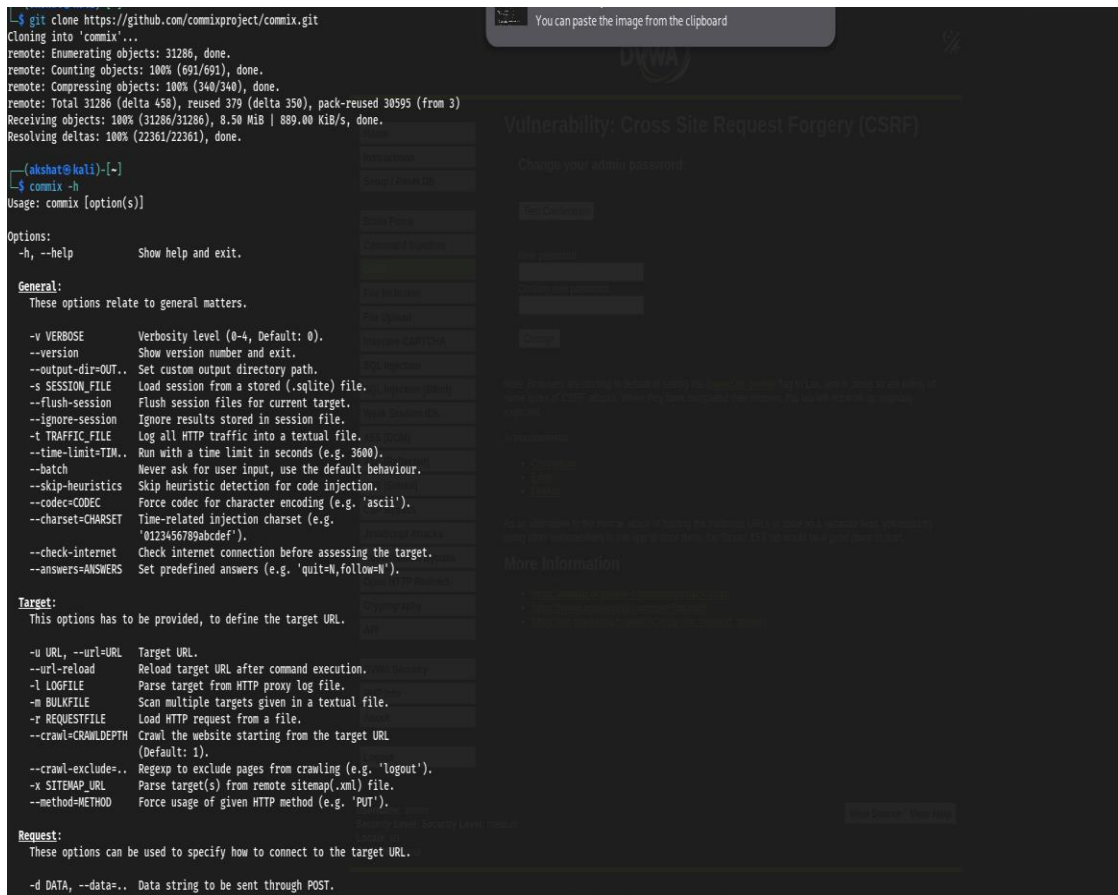


Figure 3: Commix Tool - Installation and Help Menu

The Commix (Command Injection Exploiter) tool was cloned from its official GitHub repository and its help menu was reviewed. Commix automates the detection and exploitation of OS command injection vulnerabilities, offering options for target specification, request handling, crawling, and injection technique control.

```
python3 -u http://localhost/DNA/vulnerabilities/exec/#
python3: can't open file '/home/akshat/comix.py': [Errno 2] No such file or directory

(-akshat@kali)-[~]
└─$ ls -la
total 8861532
drwx----- 47 akshat akshat    4096 Sep 24 14:36 .
drwx-r-x 3 root root    4096 May 5 16:46 ..
-rw----- 1 akshat akshat 10251 May 17 20:21 2026-05-17-14-51-12.094-VirtualBoxWM-6743.log
-rw----- 1 akshat akshat 6941 May 17 20:24 2026-05-17-14-51-20.094-VirtualBoxWM-15072.log
-rw----- 1 akshat akshat 3630 May 17 20:59 2026-05-17-15-29-08.040-VirtualBoxWM-19420.log
-rw----- 1 akshat akshat 3630 May 23 19:52 2026-05-23-14-22-59.074-VirtualBoxWM-80320.log
-rw----- 1 akshat akshat 3799 May 23 20:14 2026-05-23-14-44-43.059-VirtualBoxWM-102321.log
-rw----- 1 akshat akshat 3632 May 23 20:26 2026-05-23-14-56-14.019-VirtualBoxWM-104285.log
-rw----- 1 akshat akshat 3632 May 23 22:35 2026-05-23-17-05-50.059-VirtualBoxWM-106732.log
-rw----- 1 akshat akshat 3632 May 24 05:06 2026-05-23-23-36-33.023-VirtualBoxWM-162909.log
-rw----- 1 akshat akshat 3852 May 25 01:56 2026-05-24-20-26-18.093-VirtualBoxWM-64449.log
-rw----- 1 akshat akshat 3634 Jun 20 02:00 2026-06-19-20-30-362.089-VirtualBoxWM-65943.log
-rw----- 1 akshat akshat 3635 Jun 20 02:07 2026-06-19-20-37-50.092-VirtualBoxWM-70362.log
-rw----- 1 akshat akshat 3632 Jun 20 04:16 2026-06-19-22-46-26.070-VirtualBoxWM-70747.log
-rw----- 1 akshat akshat 3630 Jun 20 13:51 2026-06-20-08-21-32.052-VirtualBoxWM-20979.log
-rw----- 1 akshat akshat 3637 Jul 4 20:16 2026-07-04-14-46-45.023-VirtualBoxWM-127043.log
-rw----- 1 akshat akshat 10256 Jul 4 20:17 2026-07-04-14-47-01.048-VirtualBoxWM-127128.log
-rw----- 1 akshat akshat 3631 Jul 4 20:46 2026-07-04-15-16-25.081-VirtualBoxWM-130827.log
-rw----- 1 akshat akshat 3627 Jul 6 16:49 2026-07-06-11-19-05.018-VirtualBoxWM-40231.log
-rw----- 1 akshat akshat 3627 Jul 6 21:55 2026-07-06-16-25-03.834-VirtualBoxWM-47560.log
-rw----- 1 akshat akshat 3795 Jul 6 21:57 2026-07-06-16-27-47.059-VirtualBoxWM-51070.log
-rw----- 1 akshat akshat 10755 Jul 7 16:07 2026-07-07-10-37-57.065-VirtualBoxWM-54038.log
-rw----- 1 akshat akshat 13809 Aug 16 00:43 2026-08-16-03-04-38.065-VirtualBoxWM-10191.log
-rw----- 1 akshat akshat 7053 Aug 18 08:30 2026-08-18-02-46-39.015-VirtualBoxWM-15283.log
-rw----- 1 akshat akshat 3631 Aug 18 15:04 2026-08-18-09-34-45.025-VirtualBoxWM-66727.log
-rw-r--r-- 1 akshat akshat 2155782184 Sep 24 14:53 2026-09-24 14-05-36.mp4
-rwxr-x-x 1 akshat akshat 7774 May 13 16:56 42941.py
-rwxr-x-x 1 akshat akshat 3447 May 13 16:29 42944.py
drwxr-x-- 2 akshat akshat 4096 May 23 23:56 .android
drwxr-x-- 3 akshat akshat 4096 May 13 19:42 .antigravity
-rw-r--r-- 1 akshat akshat 220 May 5 16:46 .bash_logout
-rw-rw-r-- 1 akshat akshat 5840 May 22 22:22 .bashrc
-rw-r--r-- 1 akshat akshat 5782 Jun 21 02:02 .bashrc
-rw-r--r-- 1 akshat akshat 3526 May 5 16:46 .bashrc.original
drwx----- 12 akshat akshat 4096 Sep 22 13:09 .BurpSuite
drwxr-x-x 4 root root 4096 Aug 18 13:25 Burpsuite-Professional
drwx----- 32 akshat akshat 4096 Aug 30 18:34 .cache
drwxr-x-- 7 akshat akshat 4096 Sep 24 14:36 comix
drwxr-x-x 40 akshat akshat 4096 Sep 20 19:17 .config
drwxr-x-- 2 akshat akshat 4096 Sep 22 10:50 .config
drwx----- 5 akshat akshat 4096 Jul 17 16:17 .copilot
-rw-rw-r-- 1 akshat akshat 2392 Jun 13 06:41 crt_subdomains.txt
drwxr-x-- 3 akshat akshat 4096 May 30 07:11 .deepFace
drwxr-x-- 3 akshat akshat 4096 May 22 22:05 Desktop
-rw-r--r-- 1 akshat akshat 35 Jun 7 09:42 .dmrc
drwxr-x-- 2 akshat akshat 4096 May 5 16:55 Documents
drwxr-x-- 3 akshat akshat 4096 May 22 18:11 .dotnet
drwxr-x-- 8 akshat akshat 13288 Feb 24 00:21 Downloads
```

Figure 4: Commix - Initial Target Configuration

An initial attempt to run Commix against the DVWA Command Injection endpoint failed due to an incorrect file path, which was then corrected. This step illustrates the practical troubleshooting involved in pointing automated exploitation tools at the correct vulnerable endpoint.

```

python3 commix.py -u http://localhost/DVWA/vulnerabilities/exec/
y4.2.dev161
https://commixproject.com
(commixproject)

Automated All-in-One OS Command Injection Exploitation Tool
Copyright © 2014-2026 Anastasios Stasinopoulos (@ancst)

( ) Legal disclaimer: Attacking targets without prior mutual consent is illegal. Obeying applicable laws is your responsibility and commix developers assume no liability.

[14:54:39] [info] Testing connection to the target URL.
Got a 302 redirect to 'http://localhost/DVWA/login.php'. Do you want to follow? [Y/n] n
[14:54:49] [info] Checking whether some kind of WAF/IPS is protecting the target.
You have not declared any cookie(s), but the server wants to set its own ('security=impossible;PHPSESSID=d00ec45b555...7db597b21a'). Do you want to use those? [Y/n] y
[14:55:01] [critical] No parameter(s) found for testing in the provided data (e.g. GET parameter 'id' in 'www.site.com/index.php?id=1'). Re-run with '--crawl=2'.

(akshat@kali)~$ commix
Usage: commix [option(s)]

Options:
  -h, --help          Show help and exit.

General:
  These options relate to general matters.

  -v VERBOSE          Verbosity level (0-4, Default: 0).
  --version            Show version number and exit.
  --output-dir=OUT...  Set custom output directory path.
  -s SESSION_FILE      Load session from a stored (.sqlite) file.
  --flush-session      Flush session files for current target.
  --ignore-session     Ignore results stored in session file.
  -t TRAFFIC_FILE       Log all HTTP traffic into a textual file.
  --time-limit=TIM...  Run with a time limit in seconds (e.g. 3600).
  --batch              Never ask for user input, use the default behaviour.
  --skip-heuristics    Skip heuristic detection for code injection.
  --codec=CODEC        Force codec for character encoding (e.g. 'ascii').
  --charset=CHARSET    Time-related injection charset (e.g. '0123456789abcdef').
  --check-internet     Check internet connection before assessing the target.
  --answers=ANSWERS    Set predefined answers (e.g. 'quit=N, follow=N').

Target:
  This options has to be provided, to define the target URL.

  -u URL, --url=URL    Target URL.
  --url-reload         Reload target URL after command execution.
  -l LOGFILE            Parse target from HTTP proxy log file.
  -m BULKFILE           Scan multiple targets given in a textual file.
  -r REQUESTFILE        Load HTTP request from a file.
  --crawl=CRAWLDEPTH   Crawl the website starting from the target URL
  
```

Figure 5: Commix - Automated Scan Against DVWA

Commix was executed against the DVWA Command Injection endpoint (<http://localhost/DVWA/vulnerabilities/exec/>). The tool tested the connection, handled the session cookie, and reported that no injectable parameter was found in the request as submitted, indicating that the scan needed additional crawl/parameter configuration to reach the vulnerable input field.

The screenshot shows the Burp Suite interface with the Intercept feature enabled. A GET request is captured from the URL `http://localhost/DVWA/vulnerabilities/csrf/?password_new=1234&password_conf=&Change=Change`. The request details are visible in the Request tab, showing headers like `Host: localhost`, `User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36`, and `Cookie: PHPSESSID=89fc08292c0ff34cdf60ca7955936b4; security=medium`. The Inspector panel on the right shows the request attributes, query parameters, body parameters, cookies, and headers. The DVWA application is visible in the background, showing the 'Vulnerability: Cross Site Request Forgery' page with a 'Change your admin password' form.

Figure 6: Burp Suite - Intercepting a CSRF Password Change Request

Burp Suite's Proxy > Intercept feature was used to capture a live GET request generated by DVWA's Cross-Site Request Forgery (CSRF) module while changing the admin password. The intercepted request shows the `password_new` and `password_conf` parameters being sent in plain query-string form, along with session cookies and headers - illustrating exactly how a CSRF attack can forge this same request from an external page without the user's knowledge, since no anti-CSRF token is present in this request.

5. Conclusion / Learnings

This phase of Task 3 provided practical exposure to Command Injection vulnerabilities and to the interception-based analysis of a CSRF-vulnerable request. Manually exploiting DVWA's Command Injection module using a Netcat listener demonstrated how unsanitised user input can be chained with legitimate OS commands to gain command execution on the underlying server. Working with Commix highlighted how automated exploitation tools operate - handling sessions, following redirects, and searching for injectable parameters - and also showed that such tools still require correctly scoped targets and parameters to succeed, reinforcing the value of understanding the manual technique first.

Analysing the DVWA CSRF module with Burp Suite made the underlying weakness of the vulnerability concrete: the password-change request relied purely on the victim's authenticated session, with no unique, unpredictable anti-CSRF token to validate that the request was intentionally submitted by the user. This reinforced why token-based protection (synchroniser tokens) is the standard mitigation for CSRF.

The remaining sub-tasks of Task 3 - SQL Injection, XSS, File Inclusion, Intruder-based fuzzing, and security header analysis - will be carried out and documented next to complete the full OWASP Top 10 coverage planned for this task, strictly within the authorised DVWA lab environment.